

Linux Class 12 - Task

wget and curl, deleting empty files, and changing permissions with find

Submitted by: Shivakumar Muradi

Date: 6 October 2026

Task 1 - Difference between wget and curl

Both commands get things from the internet. To see the difference I used the same small website, example.com, with both of them.

First wget:

```
wget https://example.com
```

```
ls
```

```
head -5 index.html
```

A screenshot of a terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/task12'. The terminal shows the following commands and output:
1. `wget https://example.com`: The output shows the process of resolving the domain, connecting to 104.20.23.154 on port 443, receiving a 200 OK status, and saving the file as 'index.html'. A progress bar indicates 577 bytes downloaded at 1.60 MB/s.
2. `ls`: The output shows 'index.html' in the current directory.
3. `head -5 index.html`: The output shows the first five lines of the HTML document, including the doctype, meta tags, and the main content paragraph.
The terminal window has a dark background with green and white text. The window title bar shows standard Linux window controls (minimize, maximize, close).

Screenshot 1: wget downloads the page and saves it as index.html.

wget printed a lot of lines. It found the address, connected, got 200 OK which means the page is fine, and at the end it said index.html saved. When I ran ls the file was really there. head showed me that it is the HTML code of the page.

Now curl with the same website:

```
curl https://example.com
ls
curl -o page.html https://example.com
ls
```

```
Shiva@DESKTOP-MRCVEKT: ~/task12
Shiva@DESKTOP-MRCVEKT:~/task12$ curl https://example.com
<!doctype html><html lang=en><head><meta charset=utf-8><link rel=icon href=data:,><meta name=
=viewport content="width=device-width,initial-scale=1"><title>Example Domain</title><style>h
tml{color-scheme:light dark;background:light-dark(#eee,#222)}body{font:16px/1.6 system-ui,sa
ns-serif;max-width:26em;margin:auto;padding:25vh 2em 2em;text-align:center}</style></head><b
ody><p>This domain is for use in documentation examples without needing permission. This is
not a service; avoid relying on it for testing and monitoring purposes.</p><script src=/s.js
></script></body></html>
Shiva@DESKTOP-MRCVEKT:~/task12$ ls
index.html
Shiva@DESKTOP-MRCVEKT:~/task12$ curl -o page.html https://example.com
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
100    577    0    577    0     0   3150      0      0      0      0     0
Shiva@DESKTOP-MRCVEKT:~/task12$ ls
index.html  page.html
Shiva@DESKTOP-MRCVEKT:~/task12$
```

Screenshot 2: curl prints the page on the screen. With -o it saves it.

curl did not save anything. It just printed the HTML straight on my screen, and ls still showed only index.html from wget. When I added -o page.html, curl showed a small table with the size and speed and then saved the file. So now I have both files.

So the main difference I saw is this: wget is made for downloading, so it saves by itself. curl is made for moving data, so it shows the data unless I tell it to save. The other differences between them are in this table.

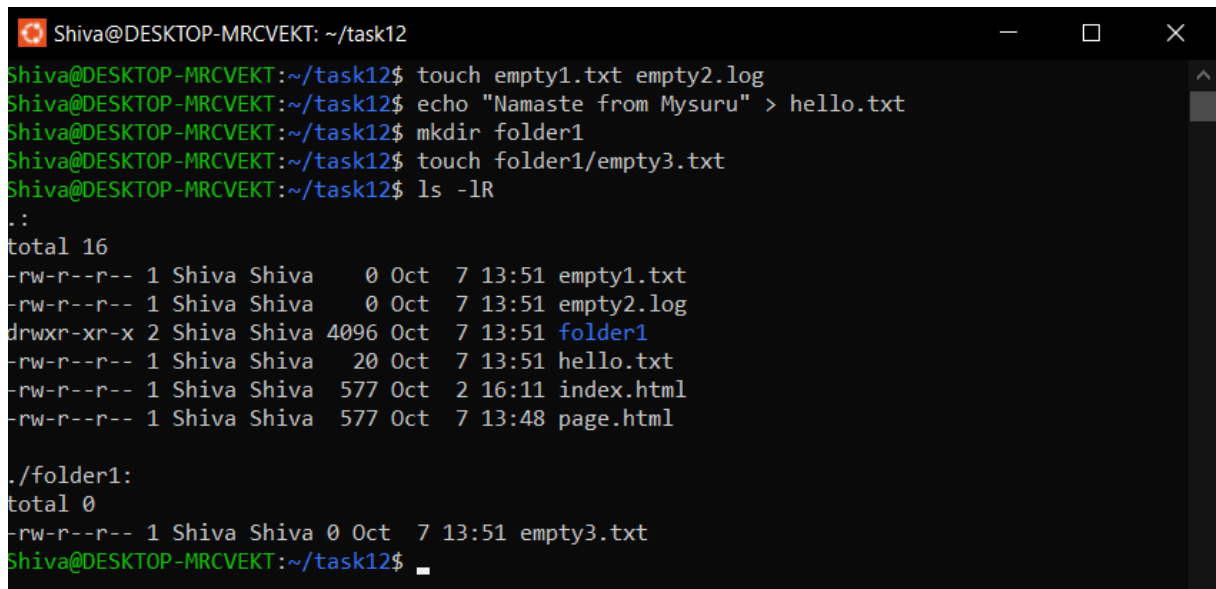
Point	wget	curl
Main job	Downloads files	Sends and receives data from a URL
What it does by default	Saves the page as a file (index.html)	Prints the page on the screen, saves nothing
Saving with my own name	wget -O name url	curl -o name url
Shows while working	Connecting, 200 OK, saved	Nothing, or a progress table when saving
Download a whole website	Yes, with -r (recursive)	No
Continue a broken download	wget -c url	curl -C - -o name url
Protocols	HTTP, HTTPS, FTP	Many more, like FTP, SFTP, SCP, SMTP
Testing APIs	Not really used for this	Yes, it can send headers and data (-H, -d, -X)

In short: if I want to download a file, wget is easier. If I want to see what a website or an API sends back, curl is better.

Task 2 - Find all the empty files and delete them

First I made some empty files to test with. touch makes an empty file. I also made one file with text in it, so I can check that it does not get deleted, and one empty file inside a folder.

```
touch empty1.txt empty2.log
echo "Namaste from Mysuru" > hello.txt
mkdir folder1
touch folder1/empty3.txt
ls -lR
```

A terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/task12' with standard window controls. It shows a series of commands and their outputs. The commands are: 'touch empty1.txt empty2.log', 'echo "Namaste from Mysuru" > hello.txt', 'mkdir folder1', 'touch folder1/empty3.txt', and 'ls -lR'. The output of 'ls -lR' shows a directory listing for the current directory and the 'folder1' subdirectory. In the current directory, there are three files with size 0 (empty1.txt, empty2.log) and one file with size 20 (hello.txt). The 'folder1' directory is empty except for 'empty3.txt' which has size 0.

```
Shiva@DESKTOP-MRCVEKT:~/task12$ touch empty1.txt empty2.log
Shiva@DESKTOP-MRCVEKT:~/task12$ echo "Namaste from Mysuru" > hello.txt
Shiva@DESKTOP-MRCVEKT:~/task12$ mkdir folder1
Shiva@DESKTOP-MRCVEKT:~/task12$ touch folder1/empty3.txt
Shiva@DESKTOP-MRCVEKT:~/task12$ ls -lR
.:
total 16
-rw-r--r-- 1 Shiva Shiva  0 Oct  7 13:51 empty1.txt
-rw-r--r-- 1 Shiva Shiva  0 Oct  7 13:51 empty2.log
drwxr-xr-x 2 Shiva Shiva 4096 Oct  7 13:51 folder1
-rw-r--r-- 1 Shiva Shiva  20 Oct  7 13:51 hello.txt
-rw-r--r-- 1 Shiva Shiva 577 Oct  2 16:11 index.html
-rw-r--r-- 1 Shiva Shiva 577 Oct  7 13:48 page.html

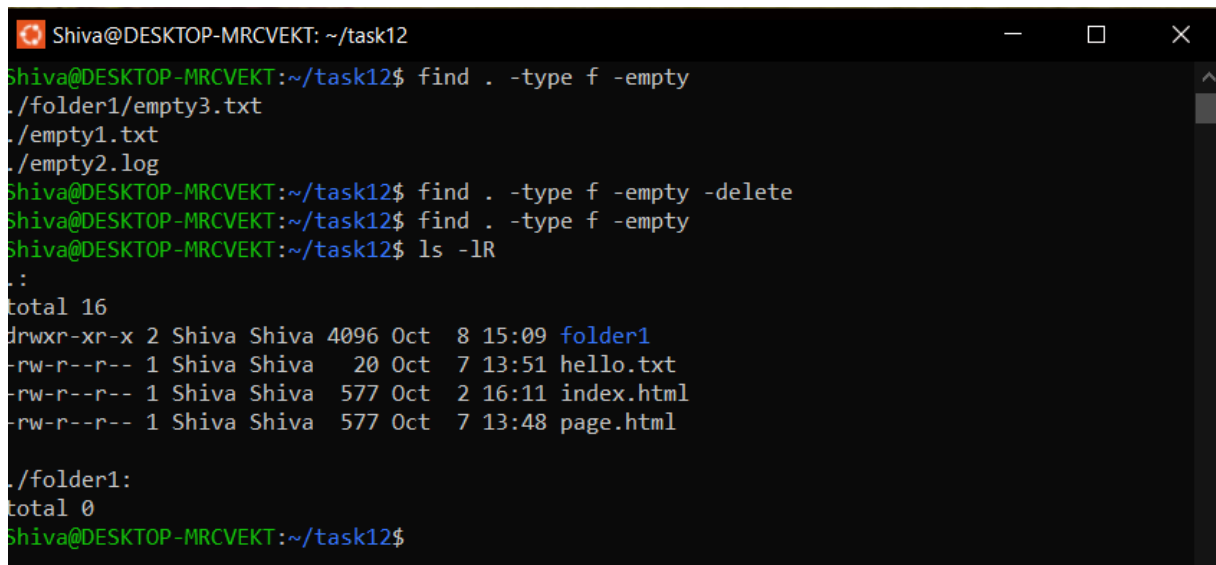
./folder1:
total 0
-rw-r--r-- 1 Shiva Shiva 0 Oct  7 13:51 empty3.txt
Shiva@DESKTOP-MRCVEKT:~/task12$
```

Screenshot 3: three empty files (size 0) and one file with text.

In ls -lR the size column shows 0 for empty1.txt, empty2.log and folder1/empty3.txt. hello.txt has 20 bytes. -R makes ls also show what is inside folder1.

Then I found the empty files first, before deleting, just to be safe. After that I ran the same command with -delete.

```
find . -type f -empty
find . -type f -empty -delete
find . -type f -empty
ls -lR
```



```
Shiva@DESKTOP-MRCVEKT: ~/task12
Shiva@DESKTOP-MRCVEKT:~/task12$ find . -type f -empty
./folder1/empty3.txt
./empty1.txt
./empty2.log
Shiva@DESKTOP-MRCVEKT:~/task12$ find . -type f -empty -delete
Shiva@DESKTOP-MRCVEKT:~/task12$ find . -type f -empty
Shiva@DESKTOP-MRCVEKT:~/task12$ ls -lR
.:
total 16
drwxr-xr-x 2 Shiva Shiva 4096 Oct  8 15:09 folder1
-rw-r--r-- 1 Shiva Shiva   20 Oct  7 13:51 hello.txt
-rw-r--r-- 1 Shiva Shiva  577 Oct  2 16:11 index.html
-rw-r--r-- 1 Shiva Shiva  577 Oct  7 13:48 page.html

./folder1:
total 0
Shiva@DESKTOP-MRCVEKT:~/task12$
```

Screenshot 4: find lists the 3 empty files, -delete removes them, and the check shows nothing.

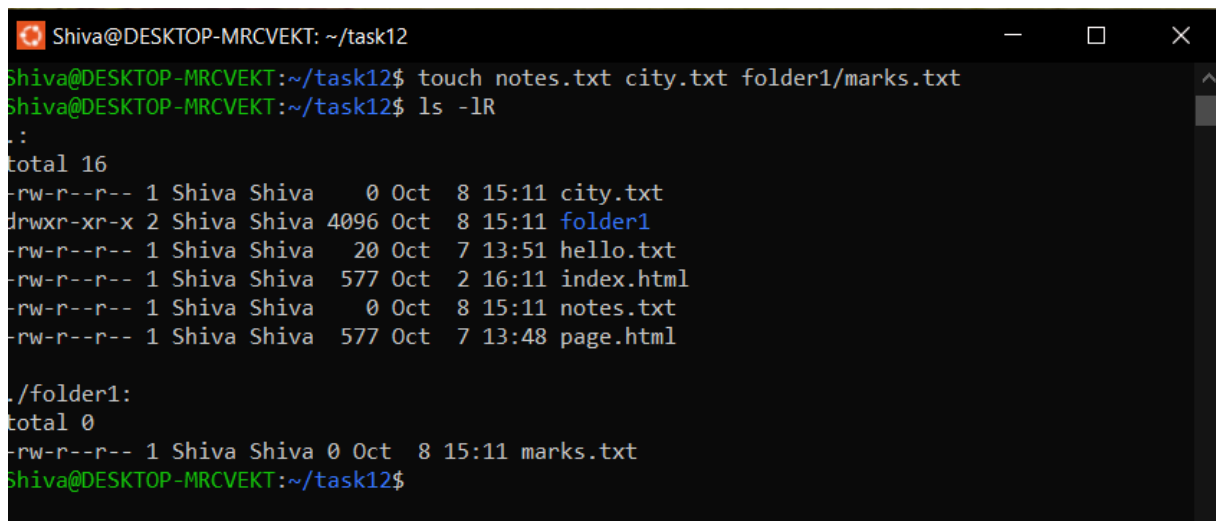
The first find showed all 3 empty files, even the one inside folder1. -delete printed nothing, which is normal. When I ran find again it showed nothing, so all the empty files are gone. ls -lR shows hello.txt and the html files are still there.

- -type f means only files. Without it, find would also delete the empty folder, because an empty folder also counts as empty.
- folder1 is still there because I used -type f. Only the file inside it was deleted.
- -delete has no undo, so running the command without -delete first is a good habit. Another way to delete is `find . -type f -empty -exec rm {} ;`

Task 3 - Find all the .txt files and change the permission to 777

For this I made a few more .txt files, again one of them inside folder1, and looked at their permissions first.

```
touch notes.txt city.txt folder1/marks.txt  
ls -lR
```



A terminal window titled 'Shiva@DESKTOP-MRCVEKT: ~/task12'. The user has executed 'touch notes.txt city.txt folder1/marks.txt' and 'ls -lR'. The output shows the following permissions:

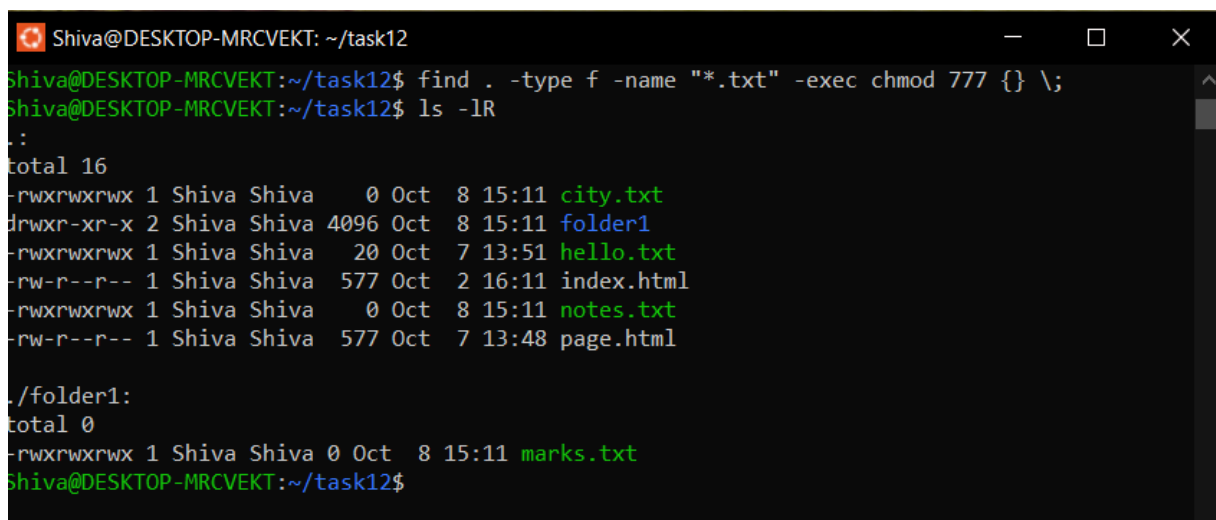
```
total 16  
-rw-r--r-- 1 Shiva Shiva 0 Oct 8 15:11 city.txt  
drwxr-xr-x 2 Shiva Shiva 4096 Oct 8 15:11 folder1  
-rw-r--r-- 1 Shiva Shiva 20 Oct 7 13:51 hello.txt  
-rw-r--r-- 1 Shiva Shiva 577 Oct 2 16:11 index.html  
-rw-r--r-- 1 Shiva Shiva 0 Oct 8 15:11 notes.txt  
-rw-r--r-- 1 Shiva Shiva 577 Oct 7 13:48 page.html  
  
./folder1:  
total 0  
-rw-r--r-- 1 Shiva Shiva 0 Oct 8 15:11 marks.txt
```

Screenshot 5: before the change, every file is -rw-r--r-- (644).

All the files start as -rw-r--r-. That is 644: the owner can read and write, everyone else can only read.

Then I used find with -exec. -exec runs a command on every file that find finds. {} is where the file name goes, and ; tells find that the command is finished.

```
find . -type f -name "*.txt" -exec chmod 777 {} ;  
ls -lR
```



The same terminal window after running 'find . -type f -name "*.txt" -exec chmod 777 {} \;'. The permissions for the .txt files have changed to -rwxrwxrwx (777), and their names are highlighted in green:

```
total 16  
-rwxrwxrwx 1 Shiva Shiva 0 Oct 8 15:11 city.txt  
drwxr-xr-x 2 Shiva Shiva 4096 Oct 8 15:11 folder1  
-rwxrwxrwx 1 Shiva Shiva 20 Oct 7 13:51 hello.txt  
-rw-r--r-- 1 Shiva Shiva 577 Oct 2 16:11 index.html  
-rwxrwxrwx 1 Shiva Shiva 0 Oct 8 15:11 notes.txt  
-rw-r--r-- 1 Shiva Shiva 577 Oct 7 13:48 page.html  
  
./folder1:  
total 0  
-rwxrwxrwx 1 Shiva Shiva 0 Oct 8 15:11 marks.txt
```

Screenshot 6: after the change, all four .txt files are -rwxrwxrwx (777).

Now city.txt, hello.txt, notes.txt and folder1/marks.txt are -rwxrwxrwx and their names turned green, because green means the file can be executed. index.html and page.html did not change, because they do not end with .txt. So find picked only the right files, even inside the folder.